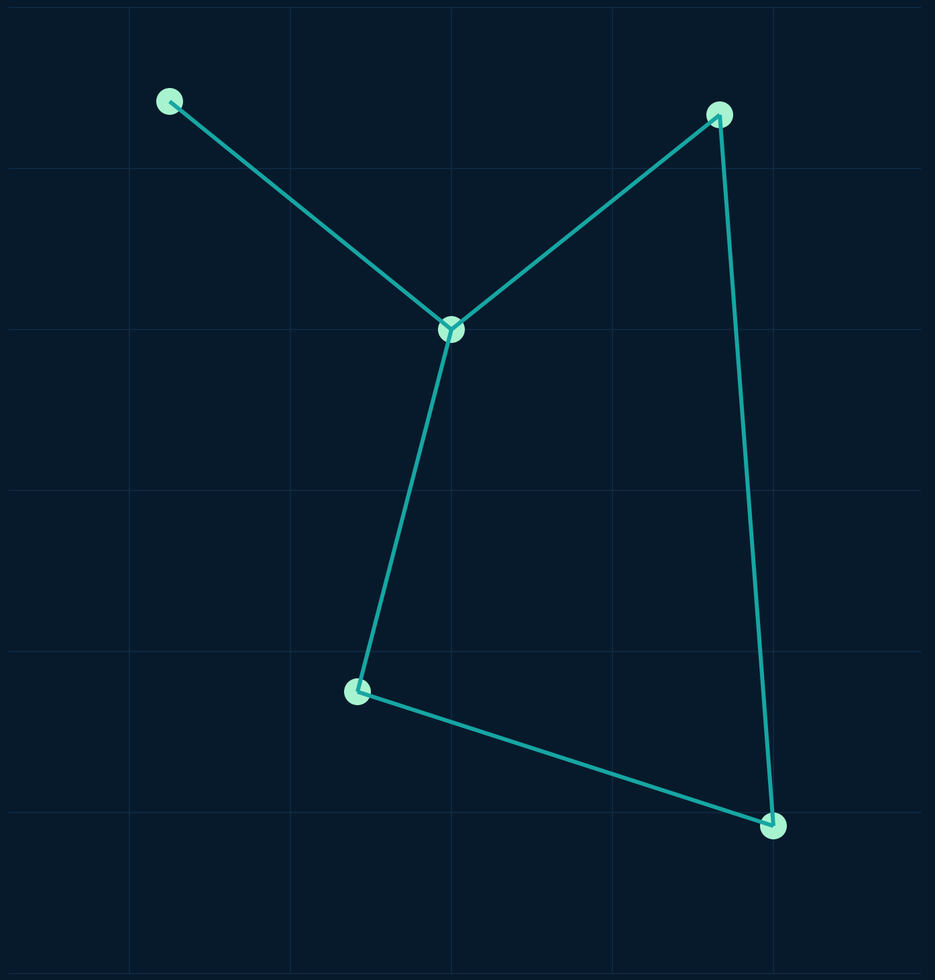
FULL-STACK BANKING APPLICATION

Connecting our Angular frontend to a Spring Boot backend and PostgreSQL database



Presented by Group One!

PROJECT CONTEXT

Project Overview

Bank of CLI Part 3 brings the previous phases of the project together into a complete full-stack banking application. We connected the Angular frontend to a Spring Boot REST API, replaced mock data with real HTTP communication, persisted account and transaction data in PostgreSQL, added security with CSRF protection, and prepared the application for deployment on AWS.

Users can manage their account, perform banking transactions, and view real transaction history through the web application.

Tech Stack

Frontend



Backend



Database



Security



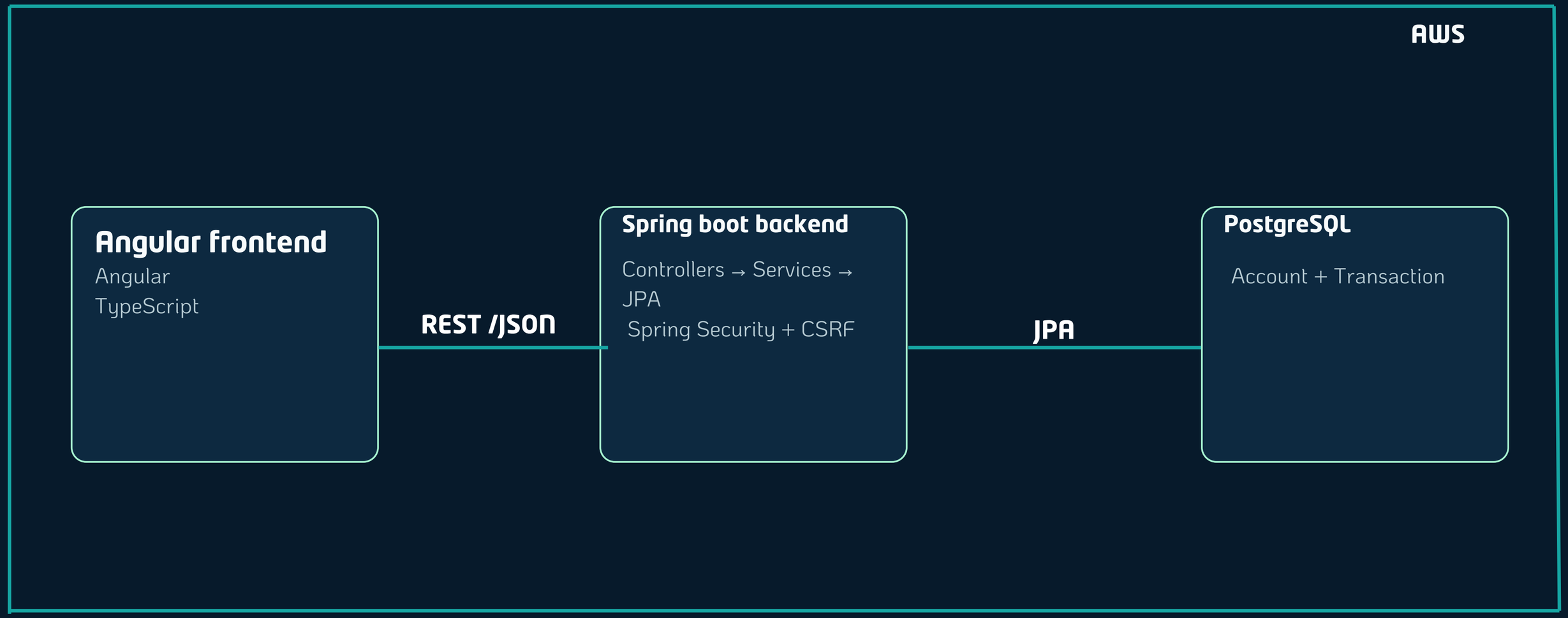
Cloud



Version Control



System Architecture



Bank Service and Atomicity

Service Interaction

01

Bank Service

Handles the deposit, withdraw, and transfer logic.

02

Account Service

Updates the balance.

02

Transaction Service

Adds a new transaction to the database.

Atomicity

```java
@Transactional
public TransactionResponseDTO deposit(DepositDTO depositDTO) {
    Integer accountId = depositDTO.getAccountId();
    BigDecimal amount = depositDTO.getAmount();

    TransactionWriteDTO transactionWriteDTO = new TransactionWriteDTO();
    transactionWriteDTO.setType(TransactionType.DEPOSIT);
    transactionWriteDTO.setAmount(amount);
    transactionWriteDTO.setAccountId(accountId);
    transactionWriteDTO.setRelatedAccountId(relatedAccountId: null);
    transactionWriteDTO.setDescription(depositDTO.getDescription());

    accountService.deposit(accountId, amount);
    Transaction transaction = transactionService.insertTransaction(transactionWriteDTO);

    return mapTransactionResponseDTO(transaction);
}
```

```java
@Transactional
public Account deposit(int id, BigDecimal amount){ ···
```

```java
@Transactional
public Transaction insertTransaction(TransactionWriteDTO transaction) { ···
```

Transfer

Transfers move money, all or nothing

Request

POST/api/transactions/transfer

BankService.transfer()

1. Block same-account transfers
2. Withdraw from the sender
3. Deposit to the recipient
4. Insert one TRANSFER row

Response

201 Created
TransactionResponseDTO

Atomic : If any step fails, @Transactional rolls back both balances and the transaction row.
Errors via GlobalExceptionHandler :
Same account: 400
Insufficient funds: 400
Amount ≤ 0: 400
Unknown account: 404

Authentication & Security

Session-Based Auth

- Each login creates a server-side session and the browser holds an HttpOnly Cookie.
- For each request, Spring looks up the session and confirms if the user is Authenticated.
- Each POST/PUT/DELETE request must carry a CSRF token.

Why Sessions and not JWT?

- A JWT stays valid until it expires, while a session can be killed serverside, this is important for a banking app.
- It is built into Spring Security, making implementation simpler.
- Using HttpOnly Cookies minimizes risk of XSS attacks

Password Hashing

- Passwords are never stored in plain text, only as a BCrypt Hash.
- BCrypt is deliberiately slow to solve, so brute-force attacked are expensive.
- Identical passwords produce different hashes.

Frontend Integration

Real API Integration

- Replaced mock API with Spring Boot HTTP calls.
- App Loads accounts using the authenticated user.

Authentication & CSRF

- Integrated CSRF token handling with backend endpoints.
- Added automatic CSRF headers via Angular interceptor.
- Restored sessions on page refresh.

API Contract Alignment

- Aligned Angular models with backend DTOs.
- Updated components to work with new API responses.

TS csrfInterceptor.ts Ln 21–25

```
if (token) {
  apiRequest = apiRequest.clone({
    setHeaders: { [token.headerName]: token.value },
  });
}
```

Login Flow with CSRF

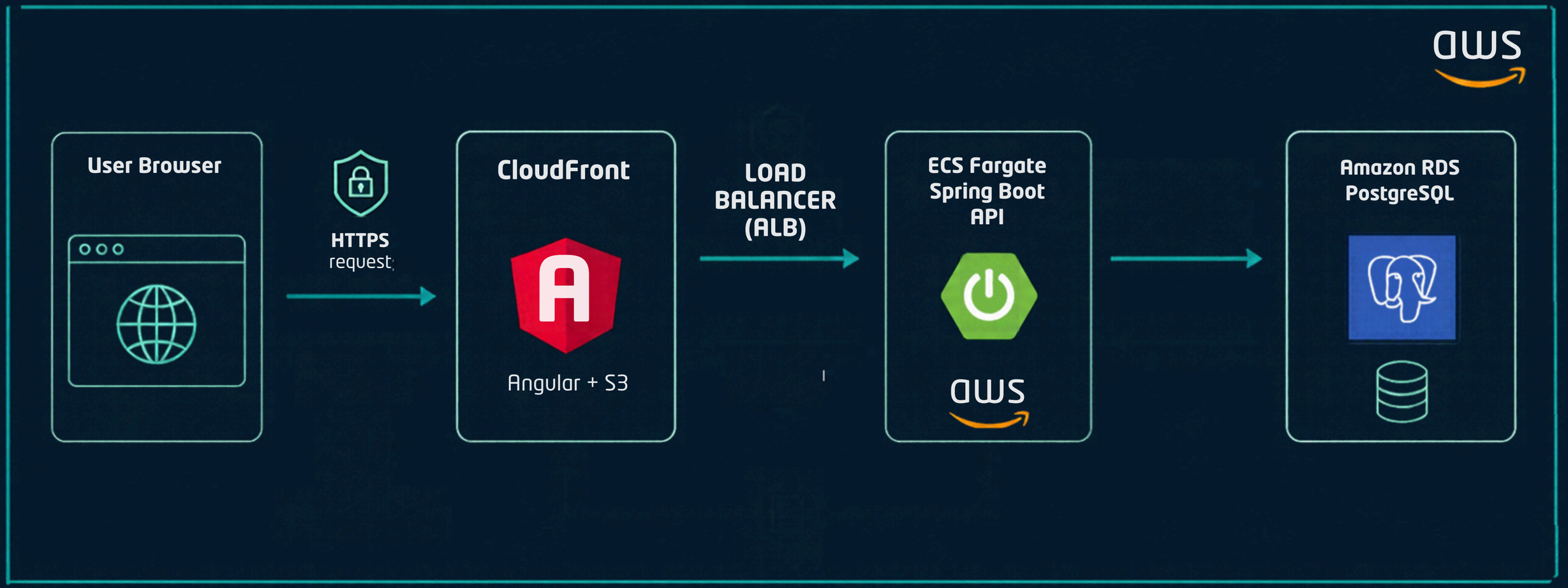
Status	Method	File	
			1. Application loads / Reload
200	GET	csrf	
401	GET	me	
			2. Login Button Clicked
200	POST	login	
200	GET	csrf	
200	GET	me	
			3. Redirected to dashboard
200	GET	5	

1. **GET /api/auth/csrf** stores the token. **GET /api/auth/me** returns 401 since we're not authenticated yet.

2. **POST /api/auth/login** sends X-XSRF-TOKEN Header with credentials. Spring replaces the cookie, so **GET /api/auth/csrf** loads the new token, then **GET /api/auth/me** loads the user.

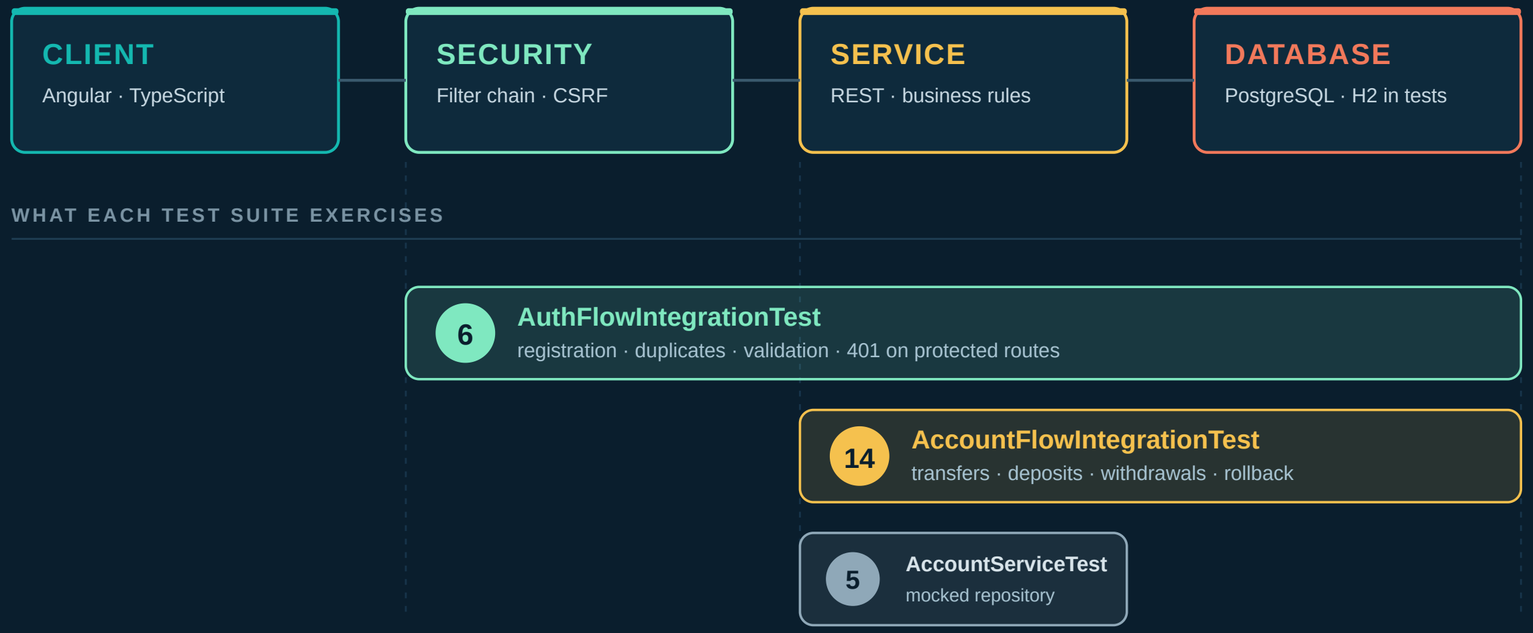
3. Populate the dashboard with **GET /api/accounts/{id}** and **GET /api/transactions/history/{id}** with the new token.

AWS Deployment



Testing & Verification

Every rule in the app is backed by a test.



01 — 26 automated tests
14 integration tests over real HTTP, 6 on authentication, 5 unit tests on the service, 1 context check.

02 — Transfers are atomic
A failed transfer asserts both balances are unchanged, including when the recipient doesn't exist.

03 — Tests never touch production data
The suite runs on in-memory H2, so it needs no credentials and can't reach RDS.

Test Driven Development

We used test-driven development. We wrote the tests first and they caught the bugs in our code.The tests caught five bugs before anyone demoed them.

Five bugs, each with a severity tag matching the Risk Management slide:

High — Withdraw added money
BankService.withdraw called deposit, so it credited the account instead of debiting it and skipped the insufficient funds check.

High — Account endpoint returned 500
The controller returned the entity, so Jackson hit a lazy collection after the session closed. It would have returned the password hash too.

High — A missing @Service stopped the app starting
TransactionService had no annotation on it, so Spring never created the bean.

Medium — New accounts had a null balance
The balance was never initialized, so the first deposit or withdrawal threw a NullPointerException.

Medium — Deposit and withdraw both returned 500
A null check was missing on relatedAccountId, which only transfers set.

DEMO

THANK YOU

A secure path from Java CLI logic to a full-stack banking experience.

Presented by Group One